

POISSON EQUATION

A CLASS PROJECT REPORT

Submitted in partial fulfillment of the requirements for
Master of Computer Applications (MCA)

Submitted By

1. Shubh Vaish	Roll No: 2025CA092
2. Saloni Gupta	Roll No: 2025CA084
3. Yusra	Roll No: 2025CA115
4. Rituja Sinha	Roll No: 2025CA079
5. Shivani Kumari	Roll No: 2025CA089
6. Shivam Pilania	Roll No: 2025CA088
7. Basudha Mahapatra	Roll No: 2025CA028
8. Sachin Shukla	Roll No: 2025CA081
9. Arpit Negi	Roll No: 2025CA020
10. Mustafa	Roll No: 2025CA060
11. Aryan Khanna	Roll No: 2025CA022

Under the Supervision of

Prof. M. M. Gore

Assistant Professor, Department of Computer Applications

Department of Computer Science and Engineering
National Institute of Technology Allahabad
Prayagraj, Uttar Pradesh, India

Semester – I (2025)

Abstract

The Poisson equation describes how potential fields behave under given source distributions and is central to physics, engineering, and computational science. In this project, the equation is solved numerically on a 2D grid using finite difference methods combined with three iterative techniques: Jacobi, Gauss–Seidel, and Successive Over-Relaxation (SOR). A C program was developed to compute potential values for an arbitrary density field and export results for visualization. Comparative analysis shows SOR converges fastest, making it an efficient approach for numerical PDE solutions.

1 Introduction

The Poisson equation relates a scalar potential function $\Phi(x, y)$ to a source density $\rho(x, y)$ through:

$$\nabla^2 \Phi(x, y) = \rho(x, y) \quad (1)$$

This form appears in electrostatics ($\nabla^2 V = -\rho/\epsilon_0$), gravitation ($\nabla^2 \Phi = 4\pi G\rho$), and many other phenomena. Analytical solutions exist only for simple geometries; hence, numerical methods are employed. This project implements finite-difference discretization with iterative solvers to approximate Φ .

2 Objectives

- Implement a C-based solver for the 2D Poisson equation.
- Compare Jacobi, Gauss–Seidel, and SOR convergence rates.
- Generate visual plots of the resulting potential fields.
- Evaluate stability and efficiency of iterative methods.

3 Methodology

The 2D Poisson equation is solved numerically by converting the continuous PDE into a discrete algebraic system. The process involves: (1) defining the physical setup and boundary conditions, (2) discretizing the domain using finite differences, (3) applying iterative solvers, and (4) testing convergence.

3.1 Discretization

A uniform $N_x \times N_y$ grid with spacing h is used, where each node (i, j) represents potential $\Phi_{i,j}$ and charge density $\rho_{i,j}$.

$$\nabla^2 \Phi(x, y) = \rho(x, y)$$

is approximated by

$$\nabla^2 \Phi_{i,j} \approx \frac{\Phi_{i+1,j} + \Phi_{i-1,j} + \Phi_{i,j+1} + \Phi_{i,j-1} - 4\Phi_{i,j}}{h^2}.$$

The iterative update formula becomes

$$\Phi_{i,j} = \frac{1}{4}(\Phi_{i+1,j} + \Phi_{i-1,j} + \Phi_{i,j+1} + \Phi_{i,j-1} - h^2 \rho_{i,j}).$$

3.2 Boundary Conditions

Dirichlet conditions fix $\Phi = 0$ along all boundaries:

$$\Phi(x, y) = 0 \quad \text{on } \partial\Omega,$$

representing a grounded region. These edge values remain constant, while only interior nodes are updated.

3.3 Iterative Solvers

Three iterative methods are implemented:

Jacobi:

$$\Phi_{i,j}^{(p+1)} = \frac{1}{4}(\Phi_{i+1,j}^{(p)} + \Phi_{i-1,j}^{(p)} + \Phi_{i,j+1}^{(p)} + \Phi_{i,j-1}^{(p)} - h^2 \rho_{i,j})$$

Uses previous iteration values; simple but slow.

Gauss–Seidel:

$$\Phi_{i,j}^{(p+1)} = \frac{1}{4}(\Phi_{i+1,j}^{(p)} + \Phi_{i-1,j}^{(p+1)} + \Phi_{i,j+1}^{(p)} + \Phi_{i,j-1}^{(p+1)} - h^2 \rho_{i,j})$$

CV Faster than Jacobi since updates are used immediately.

SOR:

$$\Phi_{i,j}^{(p+1)} = (1-\omega)\Phi_{i,j}^{(p)} + \frac{\omega}{4}(\Phi_{i+1,j}^{(p)} + \Phi_{i-1,j}^{(p+1)} + \Phi_{i,j+1}^{(p)} + \Phi_{i,j-1}^{(p+1)} - h^2 \rho_{i,j})$$

With $\omega = 1.5$, convergence is significantly faster.

3.4 Convergence

After each iteration:

$$\text{error} = \max_{i,j} |\Phi_{i,j}^{(p+1)} - \Phi_{i,j}^{(p)}|$$

If error $< 10^{-6}$, the solution is considered converged.

3.5 Algorithm Steps

1. Initialize $\Phi = 0$ and set ρ .
2. Apply $\Phi = 0$ on boundaries.
3. Iteratively update $\Phi_{i,j}$ using the chosen solver.
4. Check convergence and repeat if needed.
5. Export final Φ for visualization.

This approach provides an accurate and efficient framework for solving the 2D Poisson equation numerically.

4 Implementation

This section describes the implementation of the three iterative methods in C. Each method updates the grid values differently to approximate the solution of the 2D Poisson equation.

Common parameters used:

- Grid size: $NX = NY = 50$
- Iteration limit: 10,000
- Convergence tolerance: 10^{-6}
- Central charge source: $\rho[NX/2][NY/2] = 1$

4.1 Implementation of Jacobi Method

The Jacobi method uses only values from the previous iteration to compute new ones for all grid points. It requires an auxiliary array to store updated values before replacement.

Listing 1: Jacobi Iteration for 2D Poisson Equation

```

1 for (int iter = 0; iter < MAX_ITER; iter++) {
2     error = 0.0;
3     for (int i = 1; i < NX-1; i++) {
4         for (int j = 1; j < NY-1; j++) {
5             phi_new[i][j] = 0.25 * (
6                 phi[i+1][j] + phi[i-1][j] +
7                 phi[i][j+1] + phi[i][j-1] -
8                 h*h * rho[i][j]);
9             double diff = fabs(phi_new[i][j]
10                - phi[i][j]);
11             if (diff > error) error = diff;
12         }
13     }
14     for (int i = 1; i < NX-1; i++)
15         for (int j = 1; j < NY-1; j++)
16             phi[i][j] = phi_new[i][j];
17     if (error < TOL) break;
18 }
```

The Jacobi method is simple but converges slowly because all updates rely on the previous iteration values.

4.2 Implementation of Gauss-Seidel Method

In this approach, each newly computed value is immediately reused in subsequent calculations, which improves convergence.

Listing 2: Gauss-Seidel Iteration for 2D Poisson Equation

```

1 for (int iter = 0; iter < MAX_ITER; iter++) {
2     error = 0.0;
3     for (int i = 1; i < NX-1; i++) {
4         for (int j = 1; j < NY-1; j++) {
5             double old = phi[i][j];
6             phi[i][j] = 0.25 * (
```

```

7             phi[i+1][j] + phi[i-1][j] +
8             phi[i][j+1] + phi[i][j-1] -
9             h*h * rho[i][j]);
10            double diff = fabs(phi[i][j] -
11                old);
12            if (diff > error) error = diff;
13        }
14    }
15    if (error < TOL) break;
16 }
```

The Gauss-Seidel method generally converges faster than Jacobi since it uses updated values during computation.

4.3 Implementation of Successive Over-Relaxation (SOR) Method

The SOR method accelerates the Gauss-Seidel approach by introducing a relaxation factor ω ($1 < \omega < 2$) to overcorrect each update.

Listing 3: SOR Iteration for 2D Poisson Equation

```

1 #define OMEGA 1.5 // Relaxation factor
2
3 for (int iter = 0; iter < MAX_ITER; iter++) {
4     error = 0.0;
5     for (int i = 1; i < NX-1; i++) {
6         for (int j = 1; j < NY-1; j++) {
7             double old = phi[i][j];
8             double gs = 0.25 * (
9                 phi[i+1][j] + phi[i-1][j] +
10                phi[i][j+1] + phi[i][j-1] -
11                h*h * rho[i][j]);
12            phi[i][j] = (1 - OMEGA) * old +
13                OMEGA * gs;
14            double diff = fabs(phi[i][j] -
15                old);
16            if (diff > error) error = diff;
17        }
18    }
19    if (error < TOL) break;
20 }
```

By tuning the value of ω , the convergence rate can be significantly improved compared to the other two methods.

5 Visualization and Results

The computed data from the program was plotted using Python's matplotlib. The heatmap (Fig. ??) shows potential distribution for a central source, and the 3D surface (Fig. 3) illustrates how the potential decays smoothly from the center to the boundaries.

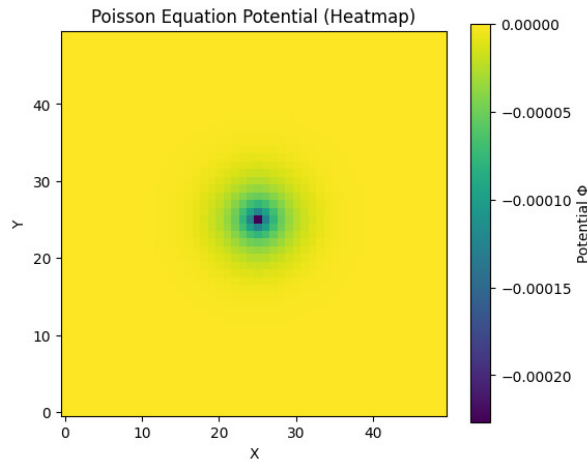


Figure 1: 2D Surface plot of computed potential field(Jacobi)

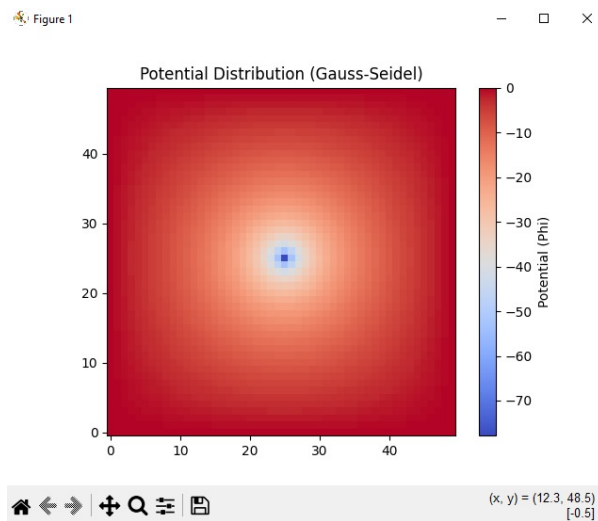


Figure 2: 2D Surface plot of computed potential field (Gauss-Seidel)

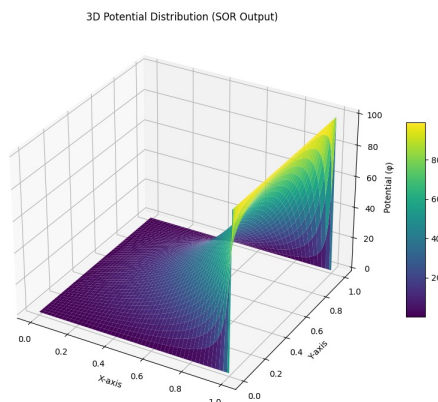


Figure 3: 3D Surface plot of computed potential field (SOR)

6 Discussion

The experiment confirms that:

- Jacobi converges slowly and is memory intensive.
- Gauss-Seidel improves performance by in-place updates.
- SOR achieves the same tolerance in far fewer iterations.

7 Individual Learnings

- **Yusra** "My primary contribution was providing strategic leadership and driving the algorithmic validation. I defined the project scope, organized the team's development timeline, and structured the final report and presentation. I focused on keeping the workflow clear and coordinated so every part of the project stayed aligned. This ensured the project reached its goal of identifying the most efficient numerical solution."
- **Mustafa:** "I learned how the SOR method helps to solve the Poisson equation faster and improved my understanding of numerical solution techniques." 1. Faster convergence. 2. less computation time 3. easy to implement 4. improves efficiency 5. useful for large system
- **Aryan khanna:** I learned the concept of the Poisson equation and how to solve it using the Jacobi Iteration Method, understanding convergence and numerical accuracy. Additionally, teamwork improved my communication, coordination, and collaborative problem-solving skills.
- **Rituja Sinha:** I understood how systematic testing boosts the accuracy and reliability of a project, and it helped me gain confidence in validating results and applying numerical methods effectively.
- **Shubh Vaish:** Learned about the basics of the Gauss-Seidel iterative method and its advantages, such as being faster than the Jacobi method and suitable for large systems of linear equations.
- **Arpit Negi:** Studied the Gauss-Seidel algorithm and realized how such numerical methods establish the relationship between weight and potential.
- **Sachin Shukla:** Learned how to solve the Poisson equation using the SOR method in C, store computed results in files, and visualize data using Python for 2D/3D plotting and analysis of potential distribution.

- **Basudha Mahapatra:** I conducted research on the Poisson equation, exploring its function and importance in physics. Additionally, I learned the process of report writing and prepared the report for our project
- **Shivam Pilania:** Understood how graphs and visuals enhance comprehension of iterative methods, helping to track convergence and solution improvement step by step.
- **Shivani Kumari:** Implemented the Jacobi iteration method for the 2D Poisson equation, gaining hands-on experience in iterative solvers, error analysis, and improving technical collaboration skills.
- **Saloni Gupta:** I learned how to verify the solution of the Poisson equation. I understood how to check whether the results converge properly. This process made me more confident in validating numerical outputs. Overall, it strengthened my understanding of numerical accuracy and stability.

8 Conclusion

The project successfully demonstrated numerical solution of the 2D Poisson equation using iterative solvers. Among all tested methods, SOR proved most efficient, achieving fast convergence with minimal computational cost. This lays groundwork for solving larger or three-dimensional PDEs in future.

9 Future Scope

- Extend to 3D Poisson problems.
- Add GPU parallelization for faster iteration.
- Explore adaptive mesh refinement.
- Include visualization directly from C using OpenGL.